PES UNIVERSITY
100 feet Ring Road, BSK 3rd Stage, Bengaluru 560085

Department of Computer Science and Engineering
(AI & ML)
B. Tech. CSE - 5th Semester
Aug – Dec 2025

UE23CS351A
DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT on

ClubConnect: Club Event Management & Registration System

Submitted by:

SRN	Name	Section
PES1UG24AM085	Dhriti Kamani	B
PES1UG24AM093	Disha Lokesh	B

Class of Prof. ***Smitha***
Lab Faculty: Smitha

Abstract

Student clubs in a university announce and manage events through scattered channels such as social media, messaging groups, forms and spreadsheets. Students cannot discover events in one place, clubs cannot track registrations reliably, and administrators cannot detect venue clashes or review events centrally. ClubConnect is a database-driven platform that manages the complete event lifecycle: a club submits an event, the campus administrator approves or rejects it, approved events appear on a central calendar, students register, clubs confirm registrations and record attendance, and the system keeps an auditable record of every step.

The system uses polyglot persistence. PostgreSQL holds all strictly structured, rule-heavy data (users, clubs, events, registrations, attendance, notifications) with primary and foreign keys, CHECK/UNIQUE constraints, a GiST exclusion constraint that forbids two approved events overlapping in one venue, stored functions, triggers, views, indexes, full-text search, row-level security and database roles. MongoDB holds flexible, high-volume content (event announcements, event feedback and an activity log with TTL expiry) and provides aggregation pipelines for reports. A Next.js 16 application with Route Handlers provides role-based interfaces for Students, Club Administrators and Campus Administrators, with bcrypt password hashing, signed session tokens and environment-based configuration.

Table of Contents

Sl. No	Topic	Page No.
1.	Introduction a) Problem Statement	4
2.	User Requirements Specification- Tech Stack	5
3.	E-R Model a) List of Entities [labeling Weak ones] b) List non-simple Attributes c) ER Diagram Snapshot d) ERD from tool like draw.io	7
4.	Relational Model a) List of Relations that are not Entities in ER Model b) Schema Diagram [ensure light background image]	9
5.	SQL DDL Statements	11
6.	SQL DML Statements	15
7.	NoSQL/ Vector Database Commands	20
8.	Results a) Resulting Tables' Screenshots	23
9.	Next.js-Project Tree Structure	25
10.	Few sample Screenshots of application code and demo	26
11.	Evaluation Rubrics- Completion Status	29

Introduction

1. Problem Statement

In colleges and universities, student clubs organise events using separate platforms: social media, messaging applications, online forms and spreadsheets. Because information is scattered, students find it hard to discover upcoming events, view complete details, register efficiently and track their registrations. Clubs struggle to maintain participant records, monitor registrations, record attendance and communicate updates. Different clubs may also plan events for the same date, time or venue, causing scheduling conflicts and poor use of campus resources, and administrators lack a centralised place to review and approve events.

ClubConnect is a centralised, database-driven platform that manages the whole event lifecycle (draft → pending approval → approved/rejected → cancelled), prevents venue clashes at the database level, guarantees that two students cannot take the last seat, and gives each role a dedicated interface.

Objectives

- Provide one calendar of approved events from all clubs.
- Enforce the approval workflow and an audit trail of status changes inside the database.
- Guarantee data integrity: no double registration, no overbooking, no overlapping approved events in a venue.
- Provide role-based access (Student, Club Admin, Campus Admin) enforced by the database through roles and row-level security.
- Use SQL for strict data and NoSQL for flexible content, justified by the nature of the data.

Scope

In scope: authentication, clubs and membership, event submission/approval, calendar and full-text search, registration (open and manual approval), attendance, trigger-generated notifications, venue conflict detection, SQL and MongoDB reports, announcements, feedback and activity logs. Out of scope for this version: online payments, refunds and QR-code attendance (listed under future enhancements).

User Requirements Specification

Functional requirements

Role	Functionality
Student	Register/login, browse and search events, calendar view, register/cancel, track registration status, join clubs, notifications, participation history, give feedback.
Club Administrator	Create/edit events, submit for approval, approve or reject registrations, mark attendance, post announcements, view participants and club reports.
Campus Administrator	Review (approve/reject) events, detect venue conflicts, manage clubs, venues, categories and staff accounts, campus-wide reports and activity log.

Technology stack and justification

Layer	Technology and reason
Frontend	Next.js 16 / React 19 with TypeScript: server-rendered role-based pages, forms, tables, calendar and dashboards.
Backend/API	Next.js Route Handlers (/api/*) with Zod input validation; secrets only through environment variables (.env).
SQL database	PostgreSQL 15+: transactions, row locks, CHECK/UNIQUE/exclusion constraints, triggers, functions, views, full-text search (tsvector + GIN), row-level security and roles.
NoSQL database	MongoDB (driver 7.x): document store for announcements, feedback and logs; JSON-Schema validators, unique and TTL indexes, aggregation pipelines.
Security	bcryptjs password hashing, jose signed session cookies, rate limiting, least-privilege database roles.
Version control	Git + GitHub.

Justification for choosing SQL (PostgreSQL)

Registrations, approvals and seat counts are connected, rule-heavy data that must stay correct. This requires foreign keys, constraints, ACID transactions and joins. Seat counting needs a row lock so that two students cannot take the last seat; a UNIQUE (event_id, student_id) constraint prevents duplicates; and the GiST exclusion constraint on (venue_id, time range) lets the database itself refuse overlapping approved events. Reports (joins, GROUP BY/HAVING, recursive CTEs, window functions) are also natural in SQL.

Justification for choosing NoSQL (MongoDB)

Three kinds of data have a different nature: announcements (free-form posts with variable tags, no joins), feedback (one flexible document per attendee, mostly read as summaries) and the activity log (append-only, fast-growing, with action-specific metadata that expires automatically). A document model stores tags as an embedded array, avoids joins, supports a TTL index for 180-day log expiry

and offers aggregation pipelines for average ratings and tag counts. A MongoDB failure cannot affect registrations, because all strict-consistency data stays in PostgreSQL.

Honest note: PostgreSQL JSONB could also store this content; MongoDB was chosen because these three parts naturally suit documents and the course requires a second database paradigm.

How the two databases connect

- Mongo documents store the numeric event_id / user_id of the PostgreSQL rows (events are never hard-deleted).
- PostgreSQL decides authorisation first (is this user an admin of the event's club? did the student attend?); MongoDB stores the content afterwards.
- PostgreSQL commits first; logging to MongoDB never breaks the main workflow. There are no cross-database transactions.

E-R Model

a) List of Entities (weak entities highlighted)

Entity	Key attributes	Remarks
USER	user_id (PK), email, password_hash, full_name, role, is_active, created_at	Supertype for the three roles
STUDENT	user_id (PK, FK), srn, department, year_of_study, phone	1:1 specialisation of USER
CLUB	club_id (PK), name, description, contact_email, is_active	
EVENT_CATEGORY	category_id (PK), name, parent_category_id (FK)	Self-referencing hierarchy
VENUE	venue_id (PK), name, building, capacity, is_active	
EVENT	event_id (PK), title, start_time, end_time, deadline, capacity, fee, eligibility, status …	
REGISTRATION	registration_id (PK), status, registered_at …	Associative: STUDENT–EVENT
ATTENDANCE (WEAK)	registration_id (PK/FK), present, marked_by, marked_at	Weak: existence depends on a REGISTRATION (1 : 0..1)
EVENT_STATUS_HISTORY (WEAK)	history_id, event_id (FK), old_status, new_status, changed_by, changed_at	Weak: audit rows depend on an EVENT
NOTIFICATION	notification_id (PK), user_id, event_id, type, message, is_read	
CLUB_ADMIN	(club_id, user_id) PK	M:N relationship table
CLUB_MEMBER	(club_id, student_id) PK, member_role, joined_on	M:N relationship table

b) Non-simple attributes

- **Derived:** seats taken / available seats of an event (computed by seats_taken() and available_seats()); fill percentage (view).
- **Composite / multivalued handling:** event time range = (start_time, end_time) used as a tstzrange; announcement tags and feedback tags are multivalued and stored as an array inside MongoDB documents.
- **Generated:** users.role pinned in students/club_admins, events.search_vector (tsvector).

c) Relationships and cardinalities

- USER – STUDENT: 1 : 0..1 (specialisation by role).
- CLUB – EVENT: 1 : N (a club organises many events).
- VENUE – EVENT: 1 : N; EVENT_CATEGORY – EVENT: 1 : N; EVENT_CATEGORY – EVENT_CATEGORY: 1 : N (parent–child).
- STUDENT – EVENT: M : N through REGISTRATION (UNIQUE per pair).
- REGISTRATION – ATTENDANCE: 1 : 0..1.
- STUDENT – CLUB: M : N through CLUB_MEMBER; CLUB_ADMIN user – CLUB: M : N through CLUB_ADMIN.
- USER – NOTIFICATION: 1 : N; EVENT – EVENT_STATUS_HISTORY: 1 : N.

d) ER Diagram

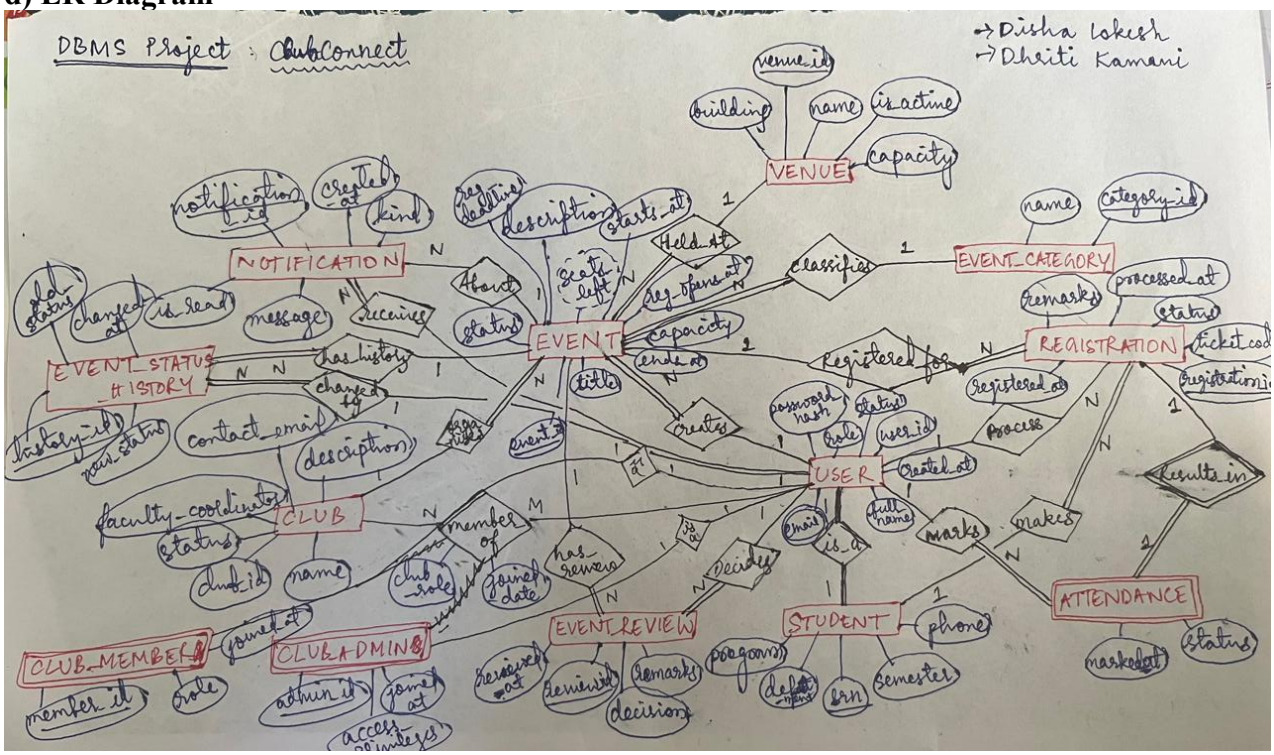


Figure 1: ER diagram of ClubConnect (draw.io)

Relational Model

a) List of relations

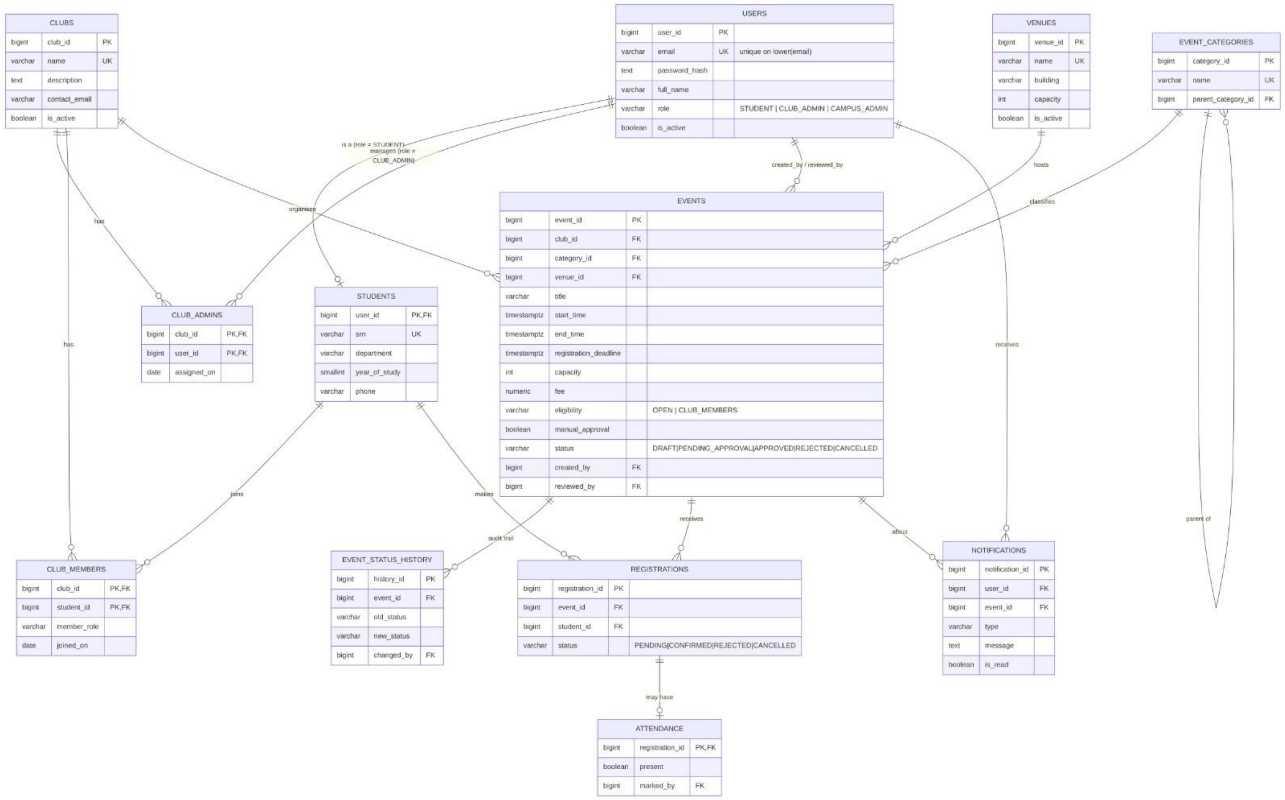
Relations that are not entities in the ER model (created from relationships):

- **registrations** – from the M:N STUDENT–EVENT relationship (with its own attributes).
- **club_members** – from the M:N STUDENT–CLUB relationship.
- **club_admins** – from the M:N CLUB_ADMIN user–CLUB relationship.
- **attendance** – from the 1:0..1 REGISTRATION–ATTENDANCE relationship (weak entity).
- **event_status_history** – audit relation written by a trigger.

Relation	Schema (PK underlined in text as PK; FK noted)
users	(user_id PK, email UNIQUE lower(), password_hash, full_name, role, is_active, created_at)
students	(user_id PK/FK→users(user_id, role), srn UNIQUE, department, year_of_study, phone)
clubs	(club_id PK, name UNIQUE, description, contact_email, is_active, created_at)
club_admins	(club_id PK/FK→clubs, user_id PK/FK→users, assigned_on)
club_members	(club_id PK/FK→clubs, student_id PK/FK→students, member_role, joined_on)
event_categories	(category_id PK, name UNIQUE, parent_category_id FK→event_categories)
venues	(venue_id PK, name UNIQUE, building, capacity, is_active)
events	(event_id PK, club_id FK, category_id FK, venue_id FK, title, description, start_time, end_time, registration_deadline, capacity, fee, eligibility, manual_approval, status, created_by FK, reviewed_by FK, … search_vector)
event_status_history	(history_id PK, event_id FK→events, old_status, new_status, changed_by FK, changed_at)
registrations	(registration_id PK, event_id FK, student_id FK, status, registered_at, decided_by FK, decided_at, cancelled_at, UNIQUE(event_id, student_id))
attendance	(registration_id PK/FK→registrations, present, marked_by FK, marked_at)
notifications	(notification_id PK, user_id FK, event_id FK, type, message, is_read, created_at)

b) Schema diagram

The relational schema diagram follows the ER diagram in Figure 1 (tables above with PK/FK links). Insert a light-background schema image (for example pgAdmin "ERD" or DBeaver diagram) below.



Constraints used

- CHECK: role domain, year_of_study 1–6, capacity > 0, fee ≥ 0, end_time > start_time, registration_deadline ≤ start_time, rejection requires a reason, email/phone formats.
- UNIQUE: email (case-insensitive), SRN, club name, one registration per student per event.
- EXCLUDE (GiST): no two APPROVED events overlap in the same venue (half-open ranges allow back-to-back events).
- Referential actions: ON DELETE CASCADE for dependent rows, RESTRICT for registrations and category parents.

5. SQL DDL Statements

Source: db/migrations/001_schema.sql (core tables), 003_views_indexes.sql (indexes and views) and db/demo/01_ddl.sql (CREATE/ALTER/DROP demonstration).

5.1 Core tables (001_schema.sql)

```
-- ClubConnect | Migration 001 | Core relational schema (PostgreSQL 15+)

CREATE EXTENSION IF NOT EXISTS btree_gist;   -- needed by the venue-overlap exclusion constraint

-- ------------------------------------------------------------------ users & roles
CREATE TABLE users (
  user_id       BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email         VARCHAR(254) NOT NULL,
  password_hash TEXT         NOT NULL,                      -- bcrypt hash, never plain text
  full_name     VARCHAR(120) NOT NULL CHECK (length(trim(full_name)) > 0),
  role          VARCHAR(20)  NOT NULL CHECK (role IN ('STUDENT','CLUB_ADMIN','CAMPUS_ADMIN')),
  is_active     BOOLEAN      NOT NULL DEFAULT TRUE,
  created_at    TIMESTAMPTZ  NOT NULL DEFAULT now(),
  CONSTRAINT users_email_format CHECK (email ~* '^[^@\s]+@[^@\s]+\.[^@\s]+$'),
  CONSTRAINT users_id_role_uq   UNIQUE (user_id, role)          -- target of the role-pinned foreign keys
below
);
CREATE UNIQUE INDEX users_email_lower_uq ON users (lower(email));

-- 1:1 specialisation. The generated "role" column + composite FK makes it impossible
-- for a students row to point at a non-student account, without needing a trigger.
CREATE TABLE students (
  user_id       BIGINT PRIMARY KEY,
  role          VARCHAR(20) GENERATED ALWAYS AS ('STUDENT') STORED,
  srn           VARCHAR(20) NOT NULL UNIQUE,                  -- university registration number
  department    VARCHAR(60) NOT NULL,
  year_of_study SMALLINT       NOT NULL CHECK (year_of_study BETWEEN 1 AND 6),
  phone         VARCHAR(15) CHECK (phone ~ '^[0-9+ -]{7,15}$'),
  FOREIGN KEY (user_id, role) REFERENCES users (user_id, role) ON DELETE CASCADE
);

-- ------------------------------------------------------------------ clubs
CREATE TABLE clubs (
  club_id       BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name          VARCHAR(100) NOT NULL UNIQUE,
  description   TEXT            NOT NULL DEFAULT '',
  contact_email VARCHAR(254) CHECK (contact_email IS NULL OR contact_email ~*
'^[^@\s]+@[^@\s]+\.[^@\s]+$'),
  is_active     BOOLEAN         NOT NULL DEFAULT TRUE,
  created_at    TIMESTAMPTZ  NOT NULL DEFAULT now()
);

-- M:N  CLUB_ADMIN users <-> clubs  (who may manage which club)
CREATE TABLE club_admins (
  club_id       BIGINT NOT NULL REFERENCES clubs ON DELETE CASCADE,
  user_id       BIGINT NOT NULL,
  role          VARCHAR(20) GENERATED ALWAYS AS ('CLUB_ADMIN') STORED,
  assigned_on DATE   NOT NULL DEFAULT current_date,
  PRIMARY KEY (club_id, user_id),
  FOREIGN KEY (user_id, role) REFERENCES users (user_id, role) ON DELETE CASCADE
);

-- M:N  students <-> clubs  (membership; used for CLUB_MEMBERS-only events)
CREATE TABLE club_members (
  club_id       BIGINT   NOT NULL REFERENCES clubs ON DELETE CASCADE,
  student_id  BIGINT   NOT NULL REFERENCES students (user_id) ON DELETE CASCADE,
  member_role VARCHAR(20) NOT NULL DEFAULT 'MEMBER' CHECK (member_role IN ('MEMBER','OFFICE_BEARER')),
  joined_on   DATE      NOT NULL DEFAULT current_date,
  PRIMARY KEY (club_id, student_id)
);

-- ------------------------------------------------------------------ reference data
-- Self-referencing hierarchy (e.g. Technical -> AI/ML) -> used by the recursive CTE report.
CREATE TABLE event_categories (
  category_id           BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
  name                  VARCHAR(60) NOT NULL UNIQUE,
  parent_category_id BIGINT REFERENCES event_categories ON DELETE RESTRICT,
  CHECK (parent_category_id IS NULL OR parent_category_id <> category_id)
);
CREATE TABLE venues (
  venue_id  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name          VARCHAR(80) NOT NULL UNIQUE,
  building  VARCHAR(80) NOT NULL,
  capacity  INT         NOT NULL CHECK (capacity > 0),
  is_active BOOLEAN     NOT NULL DEFAULT TRUE
);

-- ------------------------------------------------------------- events
-- Approval status lives here. Registration status lives in "registrations" (kept separate on purpose).
CREATE TABLE events (
  event_id              BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  club_id               BIGINT          NOT NULL REFERENCES clubs,
  category_id           BIGINT          NOT NULL REFERENCES event_categories,
  venue_id              BIGINT          NOT NULL REFERENCES venues,
  title                 VARCHAR(150) NOT NULL CHECK (length(trim(title)) > 0),
  description           TEXT            NOT NULL DEFAULT '',
  start_time            TIMESTAMPTZ  NOT NULL,
  end_time              TIMESTAMPTZ  NOT NULL,
  registration_deadline TIMESTAMPTZ  NOT NULL,
  capacity              INT             NOT NULL CHECK (capacity > 0),
  fee                   NUMERIC(8,2) NOT NULL DEFAULT 0 CHECK (fee >= 0),
  eligibility           VARCHAR(20)  NOT NULL DEFAULT 'OPEN' CHECK (eligibility IN
('OPEN','CLUB_MEMBERS')),
  manual_approval       BOOLEAN         NOT NULL DEFAULT FALSE,         -- club must approve each
registration
  status                VARCHAR(20)  NOT NULL DEFAULT 'DRAFT'
                        CHECK (status IN ('DRAFT','PENDING_APPROVAL','APPROVED','REJECTED','CANCELLED')),
  created_by            BIGINT          NOT NULL REFERENCES users,
  submitted_at          TIMESTAMPTZ,
  reviewed_by           BIGINT          REFERENCES users,
  reviewed_at           TIMESTAMPTZ,
  review_comment        TEXT,
  cancel_reason         TEXT,
  created_at            TIMESTAMPTZ  NOT NULL DEFAULT now(),
  updated_at            TIMESTAMPTZ  NOT NULL DEFAULT now(),
  search_vector         TSVECTOR GENERATED ALWAYS AS
                        (to_tsvector('english', coalesce(title,'') || ' ' || coalesce(description,'')))
STORED,
  CONSTRAINT events_time_order  CHECK (end_time > start_time),
  CONSTRAINT events_deadline    CHECK (registration_deadline <= start_time),
  CONSTRAINT events_reject_reason CHECK (status <> 'REJECTED' OR length(trim(coalesce(review_comment,'')))
> 0),
  -- Two APPROVED events may not use the same venue at overlapping times.
  -- Ranges are half-open [start,end) so back-to-back events are allowed.
  CONSTRAINT events_no_venue_overlap
        EXCLUDE USING gist (venue_id WITH =, tstzrange(start_time, end_time, '[)') WITH &&)
        WHERE (status = 'APPROVED')
);

-- Written by a trigger (see 002). Gives the approval workflow a full audit trail.
CREATE TABLE event_status_history (
  history_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  event_id   BIGINT      NOT NULL REFERENCES events ON DELETE CASCADE,
  old_status VARCHAR(20),
  new_status VARCHAR(20) NOT NULL,
  changed_by BIGINT      REFERENCES users,
  changed_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ------------------------------------------------------------- registrations & attendance
-- Seat-holding statuses: PENDING and CONFIRMED. REJECTED / CANCELLED release the seat.
CREATE TABLE registrations (
  registration_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  event_id      BIGINT          NOT NULL REFERENCES events   ON DELETE RESTRICT,
  student_id    BIGINT          NOT NULL REFERENCES students ON DELETE RESTRICT,
  status        VARCHAR(20) NOT NULL DEFAULT 'PENDING'
                  CHECK (status IN ('PENDING','CONFIRMED','REJECTED','CANCELLED')),
  registered_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  decided_by    BIGINT          REFERENCES users,
```

```
  decided_at    TIMESTAMPTZ,
  cancelled_at  TIMESTAMPTZ,
  CONSTRAINT registrations_one_per_student UNIQUE (event_id, student_id)   -- duplicate-registration guard
);

CREATE TABLE attendance (
  registration_id BIGINT PRIMARY KEY REFERENCES registrations ON DELETE CASCADE,  -- 1:0..1
  present       BOOLEAN         NOT NULL,
  marked_by     BIGINT          NOT NULL REFERENCES users,
  marked_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ---------------------------------------------------------------- notifications
-- Kept relational (as in the project synopsis). Rows are created only by triggers (see 002),
-- so application code never inserts them and duplicates cannot occur.
CREATE TABLE notifications (
  notification_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id       BIGINT          NOT NULL REFERENCES users ON DELETE CASCADE,
  event_id      BIGINT          REFERENCES events ON DELETE CASCADE,
  type          VARCHAR(30) NOT NULL CHECK (type IN
                  ('EVENT_SUBMITTED','EVENT_APPROVED','EVENT_REJECTED','EVENT_CANCELLED',
                   'REGISTRATION_PENDING','REGISTRATION_RECEIVED','REGISTRATION_CONFIRMED',
                   'REGISTRATION_REJECTED','REGISTRATION_CANCELLED')),
  message       TEXT            NOT NULL,
  is_read       BOOLEAN         NOT NULL DEFAULT FALSE,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

5.2 Indexes and views (003_views_indexes.sql)

```
-- ClubConnect | Migration 003 | Indexes and views

-- ---------------------------------------------------------------- indexes
-- (the PRIMARY KEY / UNIQUE constraints and the exclusion constraint already create their own indexes)
CREATE INDEX idx_events_status_start   ON events (status, start_time);          -- catalogue: approved +
upcoming
CREATE INDEX idx_events_club            ON events (club_id, status);            -- club dashboards
CREATE INDEX idx_events_venue_start     ON events (venue_id, start_time);       -- conflict look-ups for
PENDING events
CREATE INDEX idx_events_category        ON events (category_id);                -- category roll-up joins
CREATE INDEX idx_events_search          ON events USING gin (search_vector);    -- full-text search
CREATE INDEX idx_reg_student            ON registrations (student_id, status);   -- "my registrations"
CREATE INDEX idx_reg_event_status       ON registrations (event_id, status);    -- seat counting
CREATE INDEX idx_notif_user_unread      ON notifications (user_id, is_read, created_at DESC);
CREATE INDEX idx_club_members_student  ON club_members (student_id);
CREATE INDEX idx_history_event          ON event_status_history (event_id, changed_at);
CREATE INDEX idx_categories_parent      ON event_categories (parent_category_id);

-- ---------------------------------------------------------------- views
-- security_invoker = the caller's row-level-security policies apply to the underlying tables.

-- V1 public catalogue of published events
CREATE VIEW published_events WITH (security_invoker = true) AS
SELECT e.event_id, e.title, e.description, e.club_id, c.name AS club_name,
       e.category_id, cat.name AS category_name, e.venue_id, v.name AS venue_name, v.building,
       e.start_time, e.end_time, e.registration_deadline, e.capacity, e.fee, e.eligibility,
e.manual_approval,
       available_seats(e.event_id) AS seats_left,
        CASE WHEN now() < e.start_time THEN 'UPCOMING'
             WHEN now() < e.end_time   THEN 'ONGOING'
             ELSE 'COMPLETED' END AS lifecycle_phase,
        (now() <= e.registration_deadline AND available_seats(e.event_id) > 0) AS registration_open,
       e.search_vector
  FROM events e
  JOIN clubs c          ON c.club_id = e.club_id
  JOIN event_categories cat ON cat.category_id = e.category_id
  JOIN venues v         ON v.venue_id = e.venue_id
 WHERE e.status = 'APPROVED';

-- V2 registration / attendance figures per event
CREATE VIEW event_registration_summary WITH (security_invoker = true) AS
SELECT e.event_id, e.club_id, e.title, e.status AS event_status, e.start_time, e.capacity,
```

```
        count(*) FILTER (WHERE r.status = 'CONFIRMED') AS confirmed,
        count(*) FILTER (WHERE r.status = 'PENDING')   AS pending,
        count(*) FILTER (WHERE r.status = 'CANCELLED') AS cancelled,
        count(*) FILTER (WHERE r.status = 'REJECTED')  AS rejected,
        count(*) FILTER (WHERE a.present)               AS attended,
       round(100.0 * count(*) FILTER (WHERE r.status IN ('PENDING','CONFIRMED')) / e.capacity, 1) AS
fill_percent
  FROM events e
  LEFT JOIN registrations r ON r.event_id = e.event_id
  LEFT JOIN attendance a        ON a.registration_id = r.registration_id
 GROUP BY e.event_id;

-- V3 pairs of live events that overlap in the same venue (campus-wide schedule-conflict monitor)
CREATE VIEW venue_conflicts WITH (security_invoker = true) AS
SELECT a.venue_id, v.name AS venue_name,
       a.event_id AS event_a, a.title AS title_a, a.status AS status_a,
       b.event_id AS event_b, b.title AS title_b, b.status AS status_b,
       greatest(a.start_time, b.start_time) AS overlap_start,
       least(a.end_time, b.end_time)        AS overlap_end
  FROM events a
  JOIN events b ON b.venue_id = a.venue_id AND a.event_id < b.event_id
              AND tstzrange(a.start_time, a.end_time, '[)') && tstzrange(b.start_time, b.end_time, '[)')
  JOIN venues v ON v.venue_id = a.venue_id
 WHERE a.status IN ('PENDING_APPROVAL','APPROVED') AND b.status IN ('PENDING_APPROVAL','APPROVED');

-- V4 a student's participation history (RLS limits registrations to the caller's own rows)
CREATE VIEW student_participation_history WITH (security_invoker = true) AS
SELECT r.registration_id, r.student_id, r.status AS registration_status, r.registered_at,
       e.event_id, e.title, e.start_time, e.end_time, e.status AS event_status, e.fee,
        c.name AS club_name, v.name AS venue_name, a.present AS attended
  FROM registrations r
  JOIN events e ON e.event_id = r.event_id
  JOIN clubs c  ON c.club_id = e.club_id
  JOIN venues v ON v.venue_id = e.venue_id
  LEFT JOIN attendance a ON a.registration_id = r.registration_id;

-- V5 participant list for club admins / campus admins
CREATE VIEW club_participants WITH (security_invoker = true) AS
SELECT r.registration_id, r.event_id, e.club_id, e.title AS event_title,
       u.user_id AS student_id, u.full_name, u.email, s.srn, s.department, s.year_of_study,
       r.status, r.registered_at, a.present
  FROM registrations r
  JOIN events e   ON e.event_id = r.event_id
  JOIN students s ON s.user_id = r.student_id
  JOIN users u  ON u.user_id = s.user_id
  LEFT JOIN attendance a ON a.registration_id = r.registration_id;

-- V6 recursive category tree (depth + readable path + root)
CREATE VIEW category_tree AS
WITH RECURSIVE tree AS (
  SELECT category_id, name, parent_category_id, 0 AS depth, name::TEXT AS path, category_id AS root_id
        FROM event_categories WHERE parent_category_id IS NULL
  UNION ALL
  SELECT c.category_id, c.name, c.parent_category_id, t.depth + 1, t.path || ' > ' || c.name, t.root_id
        FROM event_categories c JOIN tree t ON c.parent_category_id = t.category_id
   WHERE t.depth < 10                          -- guard against accidental cycles
)
SELECT * FROM tree;
```

5.3 CREATE / ALTER / DROP demonstration (01_ddl.sql)

```
-- ClubConnect | Demo 01 | DDL: CREATE / ALTER / DROP, constraints, and how the database defends itself.
-- Runs as the owner. Uses a disposable table (demo_scratch) so no real table is ever dropped.
\set ON_ERROR_STOP off
\echo '== 1. Describe the real tables (constraints visible)'
\d events
\d registrations

\echo '== 2. CREATE a disposable table with PK, UNIQUE, CHECK, FK, DEFAULT'
DROP TABLE IF EXISTS demo_scratch;
CREATE TABLE demo_scratch (
  id            INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
  club_id   BIGINT NOT NULL REFERENCES clubs(club_id) ON DELETE RESTRICT,
  label         VARCHAR(30) NOT NULL UNIQUE,
  seats         INT NOT NULL DEFAULT 10 CHECK (seats BETWEEN 1 AND 500),
  created   TIMESTAMPTZ NOT NULL DEFAULT now()
);
\d demo_scratch

\echo '== 3. ALTER: add a column, add a named constraint, change a default, rename'
ALTER TABLE demo_scratch ADD COLUMN notes TEXT;
ALTER TABLE demo_scratch ADD CONSTRAINT demo_label_not_blank CHECK (length(trim(label)) > 0);
ALTER TABLE demo_scratch ALTER COLUMN seats SET DEFAULT 25;
ALTER TABLE demo_scratch RENAME COLUMN notes TO remark;
\d demo_scratch

\echo '== 4. Constraints in action: each statement below MUST fail'
INSERT INTO demo_scratch (club_id, label, seats) VALUES (1, 'ok-row', 20);              -- succeeds
(control)
INSERT INTO demo_scratch (club_id, label, seats) VALUES (1, 'ok-row', 20);              -- UNIQUE violation
INSERT INTO demo_scratch (club_id, label, seats) VALUES (1, 'big', 9999);               -- CHECK violation
INSERT INTO demo_scratch (club_id, label, seats) VALUES (99999, 'orphan', 5);           -- FK violation
INSERT INTO demo_scratch (club_id, label, seats) VALUES (1, '   ', 5);                  -- named CHECK
violation
INSERT INTO demo_scratch (club_id, label) VALUES (1, NULL);                             -- NOT NULL
violation
DELETE FROM clubs WHERE club_id = 1;                                                   -- RESTRICT (club has
events / dependants)

\echo '== 5. Real-table constraints: each must fail'
INSERT INTO users (email, password_hash, full_name, role) VALUES ('not-an-email', 'x', 'Bad Email',
'STUDENT');
INSERT INTO users (email, password_hash, full_name, role) VALUES ('x@y.com', 'x', 'Bad Role', 'SUPERUSER');
-- (events can only be inserted as DRAFT, so the CHECK constraints are demonstrated with draft inserts)
INSERT INTO events (club_id, category_id, venue_id, title, start_time, end_time, registration_deadline,
capacity, created_by)
SELECT club_id, category_id, venue_id, 'bad times', start_time, start_time - interval '1 hour', start_time,
10, created_by FROM events LIMIT 1;     -- events_time_order
INSERT INTO events (club_id, category_id, venue_id, title, start_time, end_time, registration_deadline,
capacity, created_by)
SELECT club_id, category_id, venue_id, 'bad deadline', start_time, end_time, start_time + interval '1 day',
10, created_by FROM events LIMIT 1;     -- events_deadline
INSERT INTO events (club_id, category_id, venue_id, title, start_time, end_time, registration_deadline,
capacity, created_by)
SELECT club_id, category_id, venue_id, 'zero capacity', start_time, end_time, start_time, 0, created_by
FROM events LIMIT 1;                    -- capacity > 0
UPDATE events SET capacity = 9999 WHERE event_id = 1;
-- trigger: only DRAFT events are editable
INSERT INTO students (user_id, srn, department, year_of_study) SELECT user_id, 'PESXX', 'CSE', 2 FROM users
WHERE role='CAMPUS_ADMIN' LIMIT 1;  -- role-pinned FK: only STUDENT accounts

\echo '== 6. DROP the disposable objects (controlled)'
DROP TABLE demo_scratch;
SELECT to_regclass('demo_scratch') AS demo_scratch_exists_after_drop;
```

6. SQL DML Statements

6.1 INSERT / UPDATE / DELETE (02_dml.sql)

```
-- ClubConnect | Demo 02 | DML: INSERT / UPDATE / DELETE / UPSERT / RETURNING, always inside a rolled-back
transaction.
\set ON_ERROR_STOP off
BEGIN;
\echo '== INSERT ... RETURNING (new venue and club)'
INSERT INTO venues (name, building, capacity) VALUES ('Demo Lab', 'Block D', 30) RETURNING venue_id, name,
capacity;
INSERT INTO clubs (name, description, contact_email) VALUES ('Demo Club', 'temporary', 'demo@pes.edu')
RETURNING club_id, name;

\echo '== multi-row INSERT with a sub-select'
INSERT INTO event_categories (name, parent_category_id)
SELECT v.n, (SELECT category_id FROM event_categories WHERE name = 'Technical')
  FROM (VALUES ('Demo Sub A'), ('Demo Sub B')) AS v(n) RETURNING category_id, name, parent_category_id;

\echo '== UPDATE ... RETURNING'
UPDATE venues SET capacity = capacity + 10 WHERE name = 'Demo Lab' RETURNING name, capacity;

\echo '== UPDATE using a join (FROM): deactivate the venue called Old Hall'
UPDATE venues v SET is_active = false FROM (SELECT venue_id FROM venues WHERE name = 'Old Hall') o WHERE
v.venue_id = o.venue_id RETURNING v.name, v.is_active;

\echo '== UPSERT: INSERT ... ON CONFLICT DO UPDATE'
INSERT INTO club_members (club_id, student_id, member_role)
SELECT (SELECT club_id FROM clubs WHERE name='Coding Club'), (SELECT user_id FROM users WHERE
role='STUDENT' ORDER BY user_id LIMIT 1), 'OFFICE_BEARER'
ON CONFLICT (club_id, student_id) DO UPDATE SET member_role = EXCLUDED.member_role RETURNING *;

\echo '== DELETE ... RETURNING'
DELETE FROM event_categories WHERE name LIKE 'Demo Sub%' RETURNING category_id, name;
DELETE FROM clubs WHERE name = 'Demo Club' RETURNING club_id, name;

\echo '== DELETE blocked by a foreign key (RESTRICT): a parent category that still has children cannot be
deleted'
DELETE FROM event_categories WHERE name = 'Technical';
ROLLBACK;
\echo '== after ROLLBACK nothing from this script persists:'
SELECT (SELECT count(*) FROM venues WHERE name='Demo Lab') AS demo_venues, (SELECT count(*) FROM clubs
WHERE name='Demo Club') AS demo_clubs;
```

6.2 Queries: joins, GROUP BY/HAVING, nested, correlated, recursive CTE, window (03_queries.sql)

```
-- ClubConnect | Demo 03 | SELECT: joins, aggregates, GROUP BY / HAVING, nested, correlated, recursive CTE,
window functions.
\pset border 2
\echo '== Q1 INNER JOIN (4 tables): published events with club, category and venue'
SELECT e.event_id, e.title, c.name AS club, cat.name AS category, v.name AS venue, e.start_time AT TIME
ZONE 'Asia/Kolkata' AS start_ist
  FROM events e JOIN clubs c USING (club_id) JOIN event_categories cat USING (category_id) JOIN venues v
USING (venue_id)
 WHERE e.status = 'APPROVED' ORDER BY e.start_time LIMIT 8;

\echo '== Q2 LEFT JOIN: every club with its number of approved events (clubs with none still appear)'
SELECT c.name, count(e.event_id) AS approved_events FROM clubs c LEFT JOIN events e ON e.club_id =
c.club_id AND e.status = 'APPROVED' GROUP BY c.club_id ORDER BY approved_events DESC, c.name;

\echo '== Q3 GROUP BY + HAVING: events with more than 20 confirmed registrations'
SELECT e.title, count(*) AS confirmed FROM events e JOIN registrations r USING (event_id) WHERE r.status =
'CONFIRMED' GROUP BY e.event_id HAVING count(*) > 20 ORDER BY confirmed DESC;

\echo '== Q4 NESTED (IN subquery): students who attended at least one Technical event (including sub-
categories via Q7 idea)'
SELECT DISTINCT u.full_name FROM users u WHERE u.user_id IN (
  SELECT r.student_id FROM registrations r JOIN attendance a USING (registration_id) JOIN events e USING
(event_id)
```

```
   WHERE a.present AND e.category_id IN (SELECT category_id FROM event_categories WHERE name IN
('Technical','AI/ML','Computer Vision','Web Development','Cybersecurity'))) ORDER BY 1 LIMIT 10;

\echo '== Q5 NESTED (NOT EXISTS): students who never registered for any event'
SELECT count(*) AS students_without_registrations FROM students s WHERE NOT EXISTS (SELECT 1 FROM
registrations r WHERE r.student_id = s.user_id);

\echo '== Q6 CORRELATED subquery: events whose fill % beats the average of their own club'
SELECT s.title, round(s.fill_percent,1) AS fill_percent,
       (SELECT round(avg(s2.fill_percent),1) FROM event_registration_summary s2 WHERE s2.club_id =
s.club_id AND s2.event_status='APPROVED') AS club_avg
  FROM event_registration_summary s
 WHERE s.event_status = 'APPROVED' AND s.fill_percent > (SELECT avg(s2.fill_percent) FROM
event_registration_summary s2 WHERE s2.club_id = s.club_id AND s2.event_status='APPROVED')
 ORDER BY s.fill_percent DESC;

\echo '== Q7 RECURSIVE CTE: the category tree with depth and path'
SELECT repeat('   ', depth) || name AS category, path FROM category_tree ORDER BY path;

\echo '== Q8 RECURSIVE CTE: all descendants of "Technical"'
WITH RECURSIVE sub AS (
  SELECT category_id, name, 0 AS depth FROM event_categories WHERE name = 'Technical'
  UNION ALL SELECT c.category_id, c.name, s.depth + 1 FROM event_categories c JOIN sub s ON
c.parent_category_id = s.category_id)
SELECT * FROM sub ORDER BY depth, name;

\echo '== Q9 (non-recursive) CTE: attendance rate per club, then rank them'
WITH per_club AS (
  SELECT c.name AS club, count(*) FILTER (WHERE r.status='CONFIRMED') AS confirmed, count(*) FILTER (WHERE
a.present) AS attended
       FROM clubs c JOIN events e ON e.club_id = c.club_id AND e.status='APPROVED' JOIN registrations r ON
r.event_id = e.event_id LEFT JOIN attendance a ON a.registration_id = r.registration_id
   WHERE e.start_time < now() GROUP BY c.club_id)
SELECT club, confirmed, attended, round(100.0 * attended / NULLIF(confirmed,0), 1) AS attendance_pct,
rank() OVER (ORDER BY attended::numeric / NULLIF(confirmed,0) DESC) AS rnk FROM per_club;

\echo '== Q10 WINDOW functions: running total of confirmed registrations per event over time, per club'
SELECT club, title, start_ist, confirmed, sum(confirmed) OVER (PARTITION BY club ORDER BY start_ist) AS
running_total
  FROM (SELECT c.name AS club, e.title, e.start_time AT TIME ZONE 'Asia/Kolkata' AS start_ist, count(r.*)
FILTER (WHERE r.status='CONFIRMED') AS confirmed
        FROM events e JOIN clubs c USING (club_id) LEFT JOIN registrations r USING (event_id) WHERE
e.status='APPROVED' GROUP BY c.name, e.event_id) x ORDER BY club, start_ist;

\echo '== Q11 Set operation: students registered for BOTH Battle of Bands AND Hackathon (INTERSECT)'
SELECT student_id FROM registrations r JOIN events e USING (event_id) WHERE e.title = 'Battle of Bands' AND
r.status='CONFIRMED'
INTERSECT SELECT student_id FROM registrations r JOIN events e USING (event_id) WHERE e.title LIKE '24-Hour
Hackathon' AND r.status='CONFIRMED' ORDER BY 1 LIMIT 10;

\echo '== Q12 Full-text search with ranking (GIN index on a generated tsvector)'
SELECT title, ts_rank(search_vector, q) AS rank FROM events, to_tsquery('english', 'photograph:*') q WHERE
search_vector @@ q ORDER BY rank DESC;

\echo '== Q13 Views: the five application views'
SELECT * FROM event_registration_summary ORDER BY event_id LIMIT 5;
SELECT * FROM venue_conflicts;
```

6.3 Stored function: register_for_event (transaction-safe seat booking)

```
CREATE FUNCTION register_for_event(p_event_id BIGINT) RETURNS registrations
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public, pg_temp AS $$
DECLARE uid BIGINT := app_user_id(); e events%ROWTYPE; reg registrations;
BEGIN
  IF app_role() IS DISTINCT FROM 'STUDENT' THEN PERFORM cc_fail('CC403', 'only students can register for
events'); END IF;
  SELECT * INTO e FROM events WHERE event_id = p_event_id FOR UPDATE;
  IF NOT FOUND OR e.status <> 'APPROVED' THEN PERFORM cc_fail('CC404', 'event not found or not open for
registration'); END IF;
  IF now() > e.registration_deadline THEN PERFORM cc_fail('CC409', 'registration deadline has passed'); END
IF;
  IF e.eligibility = 'CLUB_MEMBERS' AND NOT EXISTS
```

```
        (SELECT 1 FROM club_members WHERE club_id = e.club_id AND student_id = uid) THEN
        PERFORM cc_fail('CC403', 'this event is open to club members only');
  END IF;
  IF seats_taken(p_event_id) >= e.capacity THEN PERFORM cc_fail('CC409', 'event is full'); END IF;

  INSERT INTO registrations (event_id, student_id, status)
  VALUES (p_event_id, uid, CASE WHEN e.manual_approval THEN 'PENDING' ELSE 'CONFIRMED' END)
  ON CONFLICT (event_id, student_id) DO UPDATE
        SET status = EXCLUDED.status, registered_at = now(), cancelled_at = NULL, decided_by = NULL,
decided_at = NULL
        WHERE registrations.status = 'CANCELLED'
  RETURNING * INTO reg;
  IF NOT FOUND THEN PERFORM cc_fail('CC409', 'you already have a registration for this event'); END IF;
  RETURN reg;
END $$;
```

6.4 Triggers: registration notification / event validation

```
CREATE FUNCTION trg_registrations_guard() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
        IF NEW.status NOT IN ('PENDING','CONFIRMED') THEN
        PERFORM cc_fail('CC422', 'a registration must start as PENDING or CONFIRMED');
        END IF;
  ELSIF NEW.status IS DISTINCT FROM OLD.status THEN
        IF (OLD.status, NEW.status) NOT IN (
         ('PENDING','CONFIRMED'), ('PENDING','REJECTED'), ('PENDING','CANCELLED'),
         ('CONFIRMED','CANCELLED'), ('CANCELLED','PENDING'), ('CANCELLED','CONFIRMED')) THEN
        PERFORM cc_fail('CC409', format('illegal registration status change %s -> %s', OLD.status,
NEW.status));
        END IF;
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER registrations_guard BEFORE INSERT OR UPDATE ON registrations
  FOR EACH ROW EXECUTE FUNCTION trg_registrations_guard();

-- T5  registrations: notifications for student (and club admins when approval is needed)
CREATE FUNCTION trg_registrations_notify() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public, pg_temp AS $$
DECLARE ev events%ROWTYPE;
BEGIN
  IF TG_OP = 'UPDATE' AND NEW.status IS NOT DISTINCT FROM OLD.status THEN RETURN NEW; END IF;
  SELECT * INTO ev FROM events WHERE event_id = NEW.event_id;

  INSERT INTO notifications (user_id, event_id, type, message)
  VALUES (NEW.student_id, NEW.event_id,
        CASE NEW.status
            WHEN 'PENDING'   THEN 'REGISTRATION_PENDING'
            WHEN 'CONFIRMED' THEN 'REGISTRATION_CONFIRMED'
            WHEN 'REJECTED'  THEN 'REGISTRATION_REJECTED'
            WHEN 'CANCELLED' THEN CASE WHEN ev.status = 'CANCELLED' THEN 'EVENT_CANCELLED' ELSE
'REGISTRATION_CANCELLED' END
        END,
        CASE NEW.status
            WHEN 'PENDING'   THEN format('Your registration for "%s" is awaiting club approval', ev.title)
            WHEN 'CONFIRMED' THEN format('You are confirmed for "%s"', ev.title)
            WHEN 'REJECTED'  THEN format('Your registration for "%s" was not approved', ev.title)
            WHEN 'CANCELLED' THEN CASE WHEN ev.status = 'CANCELLED'
                                       THEN format('"%s" was cancelled by the organisers', ev.title)
                                       ELSE format('Your registration for "%s" was cancelled', ev.title)
END
        END);

  IF NEW.status = 'PENDING' THEN
        INSERT INTO notifications (user_id, event_id, type, message)
        SELECT ca.user_id, NEW.event_id, 'REGISTRATION_RECEIVED',
           format('A new registration for "%s" needs your decision', ev.title)
        FROM club_admins ca WHERE ca.club_id = ev.club_id;
  END IF;
```

```
  RETURN NEW;
END $$;
CREATE TRIGGER registrations_notify AFTER INSERT OR UPDATE OF status ON registrations
  FOR EACH ROW EXECUTE FUNCTION trg_registrations_notify();

-- T6  attendance: only CONFIRMED registrations can be marked
```

6.5 Role-based access control (004_roles_rls.sql excerpt)

```
-- state changes (they check role + ownership themselves).  Table owners (migration user) bypass RLS.

DO $$
DECLARE r TEXT;
BEGIN
  FOREACH r IN ARRAY ARRAY['app_student','app_club_admin','app_campus_admin'] LOOP
       IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = r) THEN EXECUTE format('CREATE ROLE %I
NOLOGIN', r); END IF;
  END LOOP;
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'clubconnect_app') THEN
       CREATE ROLE clubconnect_app LOGIN NOINHERIT;           -- password is set by scripts/migrate.mjs
from DB_APP_PASSWORD
  END IF;
END $$;

GRANT app_student, app_club_admin, app_campus_admin TO clubconnect_app;
GRANT USAGE ON SCHEMA public TO app_student, app_club_admin, app_campus_admin, clubconnect_app;

-- trigger function that was written without SECURITY DEFINER in 002
ALTER FUNCTION trg_registrations_guard() SECURITY DEFINER SET search_path = public, pg_temp;

-- ------------------------------------------------------------------ users  (password_hash is never readable
by any app role)
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
GRANT SELECT (user_id, email, full_name, role, is_active, created_at) ON users TO app_student,
app_club_admin, app_campus_admin;
GRANT UPDATE (full_name) ON users TO app_student, app_club_admin, app_campus_admin;
GRANT UPDATE (is_active) ON users TO app_campus_admin;
CREATE POLICY users_self_select   ON users FOR SELECT TO app_student, app_club_admin, app_campus_admin
USING (user_id = app_user_id());
CREATE POLICY users_self_update   ON users FOR UPDATE TO app_student, app_club_admin, app_campus_admin
  USING (user_id = app_user_id()) WITH CHECK (user_id = app_user_id());
CREATE POLICY users_campus_select ON users FOR SELECT TO app_campus_admin USING (true);
CREATE POLICY users_campus_update ON users FOR UPDATE TO app_campus_admin USING (true) WITH CHECK (true);
CREATE POLICY users_club_sees_participants ON users FOR SELECT TO app_club_admin
  USING (role = 'STUDENT' AND EXISTS (SELECT 1 FROM registrations r WHERE r.student_id = users.user_id));

-- ------------------------------------------------------------------ students
ALTER TABLE students ENABLE ROW LEVEL SECURITY;
GRANT SELECT ON students TO app_student, app_club_admin, app_campus_admin;
GRANT UPDATE (department, year_of_study, phone) ON students TO app_student;
CREATE POLICY students_self     ON students FOR SELECT TO app_student USING (user_id = app_user_id());
CREATE POLICY students_self_upd ON students FOR UPDATE TO app_student USING (user_id = app_user_id()) WITH
CHECK (user_id = app_user_id());
CREATE POLICY students_campus   ON students FOR SELECT TO app_campus_admin USING (true);
CREATE POLICY students_club     ON students FOR SELECT TO app_club_admin
  USING (EXISTS (SELECT 1 FROM registrations r WHERE r.student_id = students.user_id)
```

7. NoSQL / Vector Database Commands (MongoDB)

Data model: three collections referencing PostgreSQL ids. Retrieval is by indexed find() and aggregation pipelines; no vector search is used.

Collection	Document fields	Indexes
event_announcements	eventId, clubId, title, body, pinned, tags[], createdAt	(eventId, pinned −1, createdAt −1); (clubId, createdAt −1)
event_feedback	eventId, studentUserId, rating (1–5), comment, tags[], createdAt	UNIQUE (eventId, studentUserId); (eventId, rating)
activity_logs	actorUserId, actorRole, action, entityType, entityId, metadata{}, createdAt	(actorUserId, createdAt); (action, createdAt); (entityType, entityId); TTL on createdAt = 180 days

7.1 Collection creation, validators and indexes (src/lib/mongo-schema.mjs)

```
// MongoDB collection definitions: JSON-Schema validators + indexes.
// Shared by scripts/mongo-init.mjs (creates them) and the tests, so there is a single source of truth.
//
// Why MongoDB for these three collections (and PostgreSQL for everything else)?
//  - activity_logs: append-only, high-volume, loosely structured "metadata" per action, expires with a TTL
index.
//  - event_announcements: free-form posts with variable tags/attachments, edited and deleted by club
admins.
//  - event_feedback: one flexible document per attended student, aggregated for ratings/tag clouds.
// None of these needs joins or multi-row transactions. Registrations, payments, approvals and capacity DO
need them,
// so they stay in PostgreSQL. Mongo documents reference PostgreSQL rows by numeric id (events are never
hard-deleted).

export const ROLES = ['STUDENT', 'CLUB_ADMIN', 'CAMPUS_ADMIN', 'ANONYMOUS'];
export const LOG_RETENTION_SECONDS = 180 * 24 * 3600;

export const COLLECTIONS = {
  activity_logs: {
        validator: { $jsonSchema: {
        bsonType: 'object',
        required: ['actorUserId', 'actorRole', 'action', 'createdAt'],
      properties: {
        actorUserId: { bsonType: ['int', 'long', 'double', 'null'], description: 'PostgreSQL users.user_id
(null for failed logins)' },
        actorRole:   { enum: ROLES },
        action:        { bsonType: 'string', pattern: '^[A-Z_]{3,40}$' },
        entityType:  { bsonType: ['string', 'null'] },
        entityId:      { bsonType: ['int', 'long', 'double', 'null'] },
        metadata:      { bsonType: 'object' },
        createdAt:   { bsonType: 'date' },
        } } },
        indexes: [
        { key: { actorUserId: 1, createdAt: -1 }, name: 'actor_time' },
        { key: { action: 1, createdAt: -1 }, name: 'action_time' },
        { key: { entityType: 1, entityId: 1 }, name: 'entity' },
        { key: { createdAt: 1 }, name: 'ttl_createdAt', expireAfterSeconds: LOG_RETENTION_SECONDS },
        ],
  },
  event_announcements: {
        validator: { $jsonSchema: {
        bsonType: 'object',
        required: ['eventId', 'clubId', 'authorUserId', 'title', 'body', 'createdAt'],
      properties: {
```

```
        eventId:        { bsonType: ['int', 'long', 'double'] },
        clubId:         { bsonType: ['int', 'long', 'double'] },
        authorUserId: { bsonType: ['int', 'long', 'double'] },
        title:          { bsonType: 'string', minLength: 1, maxLength: 120 },
        body:           { bsonType: 'string', minLength: 1, maxLength: 2000 },
        pinned:         { bsonType: 'bool' },
        tags:           { bsonType: 'array', maxItems: 8, items: { bsonType: 'string', maxLength: 30 } },
        createdAt:      { bsonType: 'date' },
        updatedAt:      { bsonType: 'date' },
        } } },
        indexes: [
        { key: { eventId: 1, pinned: -1, createdAt: -1 }, name: 'event_pinned_time' },
        { key: { clubId: 1, createdAt: -1 }, name: 'club_time' },
        ],
  },
  event_feedback: {
        validator: { $jsonSchema: {
        bsonType: 'object',
        required: ['eventId', 'studentUserId', 'rating', 'createdAt'],
      properties: {
        eventId:        { bsonType: ['int', 'long', 'double'] },
        studentUserId: { bsonType: ['int', 'long', 'double'] },
        rating:         { bsonType: ['int', 'long', 'double'], minimum: 1, maximum: 5 },
        comment:        { bsonType: 'string', maxLength: 1000 },
        tags:           { bsonType: 'array', maxItems: 6, items: { bsonType: 'string', maxLength: 30 } },
        createdAt:      { bsonType: 'date' },
        updatedAt:      { bsonType: 'date' },
        } } },
        indexes: [
        { key: { eventId: 1, studentUserId: 1 }, name: 'one_feedback_per_student', unique: true },
        { key: { eventId: 1, rating: 1 }, name: 'event_rating' },
        ],
  },
};
```

7.2 CRUD (Create, Read, Update, Delete)

```
// CREATE
db.event_announcements.insertOne({eventId:7, clubId:2, title:"Bring your laptop", body:"Charger too.",
pinned:true, tags:["info"], createdAt:new Date()})
// READ
db.event_announcements.find({eventId:7}).sort({pinned:-1, createdAt:-1})
// UPDATE
db.event_announcements.updateOne({_id:ObjectId("...")},{$set:{pinned:false, body:"Updated"}})
// UPSERT (one feedback per student per event)
db.event_feedback.updateOne({eventId:7, studentUserId:12},{$set:{rating:5, comment:"Great",
tags:["fun"]}},{upsert:true})
// DELETE
db.event_announcements.deleteOne({_id:ObjectId("...")})
```

7.3 Aggregation pipelines (src/lib/mongo-pipelines.mjs)

```
// MongoDB aggregation pipelines (shared by the API and the tests).

/** Average rating, review count and rating histogram per event. */
export const feedbackByEvent = (eventIds) => [
  ...(eventIds ? [{ $match: { eventId: { $in: eventIds } } }] : []),
  { $group: { _id: '$eventId', avgRating: { avg: 'rating' }, reviews: { $sum: 1 },
              five: { $sum: { $cond: [{ eq:['rating', 5] }, 1, 0] } },
              low:  { $sum: { $cond: [{ lte:['rating', 2] }, 1, 0] } } } },
  { $project: { _id: 0, eventId: '$_id', reviews: 1, five: 1, low: 1, avgRating: { round:['avgRating', 2] }
} },
  { $sort: { avgRating: -1, reviews: -1 } },
];

/** Most frequent feedback tags (unwinds the tags array). */
export const feedbackTagCounts = (limit = 10) => [
  { unwind: 'tags' },
  { $group: { _id: '$tags', count: { $sum: 1 } } },
  { $sort: { count: -1, _id: 1 } },
```

```
  { $limit: limit },
  { $project: { _id: 0, tag: '$_id', count: 1 } },
];

/** Activity per day and action for the last N days. */
export const activityPerDay = (sinceDate) => [
  { $match: { createdAt: { $gte: sinceDate } } },
  { $group: { _id: { day: { $dateToString: { format: '%Y-%m-%d', date: '$createdAt' } }, action: '$action'
}, count: { $sum: 1 } } },
  { $sort: { '_id.day': 1, count: -1 } },
  { $project: { _id: 0, day: '$_id.day', action: '$_id.action', count: 1 } },
];

/** Most active users. */
export const topActors = (limit = 5) => [
  { $match: { actorUserId: { $ne: null } } },
  { $group: { _id: { user: '$actorUserId', role: '$actorRole' }, actions: { $sum: 1 }, lastSeen: {
max: 'createdAt' } } },
  { $sort: { actions: -1 } },
  { $limit: limit },
  { $project: { _id: 0, actorUserId: '$_id.user', actorRole: '$_id.role', actions: 1, lastSeen: 1 } },
];

/** Announcements per club, with how many are pinned. */
export const announcementsPerClub = () => [
  { $group: { _id: '$clubId', posts: { $sum: 1 }, pinned: { $sum: { cond: ['pinned', 1, 0] } }, latest: {
max: 'createdAt' } } },
  { $sort: { posts: -1 } },
  { $project: { _id: 0, clubId: '$_id', posts: 1, pinned: 1, latest: 1 } },
];
```

8. Results

a) Resulting tables' screenshots

Screenshot: users table

```
SELECT * FROM public.users
ORDER BY user_id ASC
```

Data Output | Messages | Notifications

Showing rows: 1 to 10 | Page No: 1 of 5

user_id [PK] bigint	email character varying (254)	password_hash text	full_name character varying (120)	role character varying (20)	is_active boolean	created_at timestamp with
1	admin@pes.edu	$2b$10$Jbv4rl3E87S4shL1Q1guWurGdsgDPuFcTn985w2vJW1xgWeS...	Dr. Meera Rao	CAMPUS_ADMIN	true	2026-10-25 17
2	ca.coding@pes.edu	$2b$10$U0Gj5npC.5Av7PEE/OUP/.vIm9Unk1T9junP9jg4pFN/zKI05RjVa	Arjun Nair	CLUB_ADMIN	true	2026-10-25 17
3	ca.music@pes.edu	$2b$10$N5wwI4WpJxHf56M0tLBZ8Oz1JMx9hM4NN0fIm699VlV3WN...	Kavya Menon	CLUB_ADMIN	true	2026-10-25 17
4	ca.photo@pes.edu	$2b$10$gtmygPGyN5xKGMUXwOtnHudviRqymH8m0sXerOFWsGhay8...	Rohan Das	CLUB_ADMIN	true	2026-10-25 17
5	aarav.sharma1@stu.pes.edu	$2b$10$6vfXHcrczgV.auB/qWZ/s.6qYOfkUjR7grnXk9sMuKo61QVjp5ZlC	Aarav Sharma	STUDENT	true	2026-10-25 17
6	diya.iyer2@stu.pes.edu	$2b$10$qzksZS2OmqmC6nYJ0Wk4cuBwsEz5iWyKTIApXRaX68RV7jlK...	Diya Iyer	STUDENT	true	2026-10-25 17
7	vihaan.reddy3@stu.pes.edu	$2b$10$qeKpjv1k7ASR1dySR/6Uve.FoMYs6tXCJYI9hvSE9QziK8qwVn...	Vihaan Reddy	STUDENT	true	2026-10-25 17
8	ananya.patel4@stu.pes.edu	$2b$10$lnU.UwQj5ZC5ymrd8gBrkeSAPeFiYxoq9yDsEIdgN0xLWY1U7T...	Ananya Patel	STUDENT	true	2026-10-25 17
9	aditya.gupta5@stu.pes.edu	$2b$10$DLJxQycrVw733PSD2IDLnuyhhRuO8dOn8ZanlLllQeLU8.iMWP...	Aditya Gupta	STUDENT	true	2026-10-25 17
10	ishita.nair6@stu.pes.edu	$2b$10$/k6Z637MgyRM/nB.uOiiUOn2JVglkv2vMC5o1fjPJ1oECINAWO...	Ishita Nair	STUDENT	true	2026-10-25 17

Screenshot: events table with APPROVED / PENDING rows

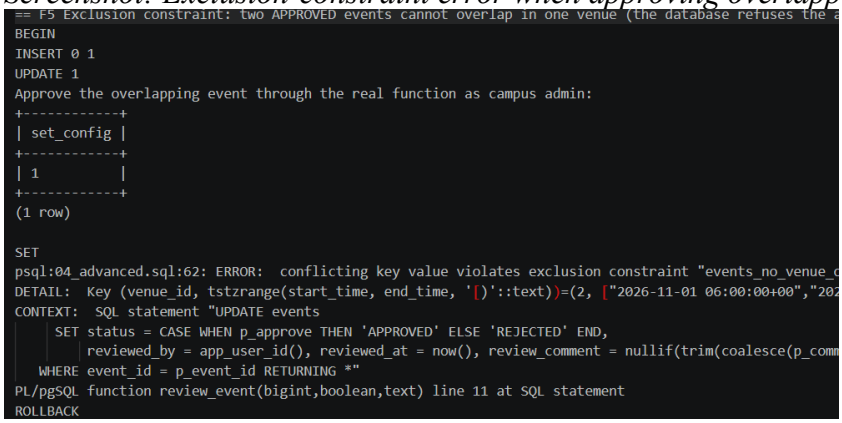
```
SELECT * FROM public.events
ORDER BY event_id ASC
```

Data Output | Messages | Notifications

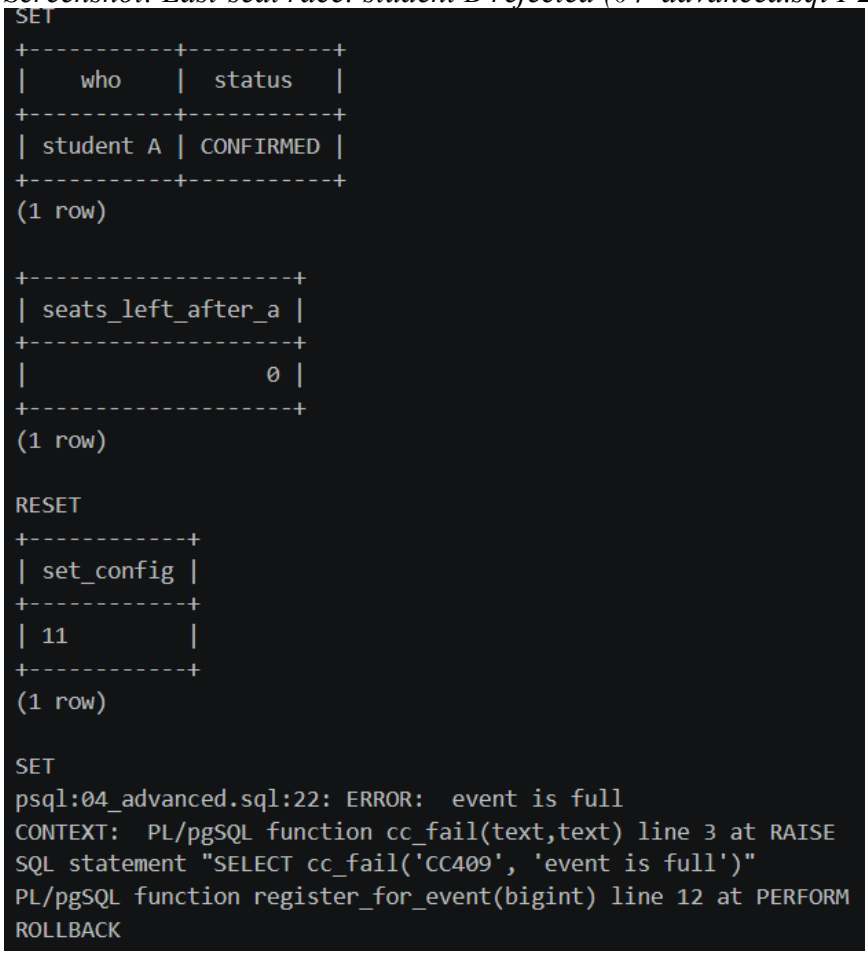
Showing rows: 1 to 15 | Page No: 1 of 1

event_id [PK] bigi	club_id bigint	categor bigint	venue_i bigint	title character varying (150)	description text	start_time timestamp wit	end_time timestamp with tir	registration_de timestamp with	capacity integer	fee numeric (8,2	eligibility character	manual_ap boolean	status character varying (20)
1	1	4	2	Intro to Web Dev Boot...	Hands-on HTML, ...	2026-10-0...	2026-10-05 13...	2026-10-04...	60	0.00	OPEN	false	APPROVED
2	2	5	4	AI/ML Paper Reading ...	We read and disc...	2026-10-1...	2026-10-13 19...	2026-10-12...	40	0.00	OPEN	true	APPROVED
3	3	8	5	Open Mic Evening	Sing, play or just ...	2026-10-1...	2026-10-17 21...	2026-10-16...	200	50.00	OPEN	false	APPROVED
4	4	9	5	Sunrise Photo Walk	Golden-hour phot...	2026-10-2...	2026-10-20 08...	2026-10-19...	40	0.00	OPEN	false	APPROVED
5	1	6	3	Cybersecurity CTF	Capture-the-flag ...	2026-10-2...	2026-10-29 16...	2026-10-28...	6	0.00	OPEN	false	APPROVED
6	2	7	4	Computer Vision Wor...	Object detection ...	2026-10-3...	2026-10-31 17...	2026-10-30...	30	0.00	CLUB_...	true	APPROVED
7	4	9	2	Portrait Photography ...	Lighting and co...	2026-11-0...	2026-11-01 13...	2026-10-31...	50	150.00	OPEN	false	APPROVED
8	4	3	3	Last Seat Demo Work...	Only ONE seat le...	2026-11-0...	2026-11-02 17...	2026-11-01...	3	0.00	OPEN	false	APPROVED
9	1	1	1	24-Hour Hackathon	Build something ...	2026-11-0...	2026-11-04 09...	2026-11-02...	150	100.00	OPEN	false	APPROVED
10	3	8	1	Battle of Bands	Campus bands c...	2026-11-0...	2026-11-08 21...	2026-11-07...	250	75.00	OPEN	false	APPROVED
11	1	4	2	Rust for Beginners	Overlaps the Por...	2026-11-0...	2026-11-01 14...	2026-10-31...	40	0.00	OPEN	false	PENDING_APPROVAL
12	3	3	4	Music Production Wo...	Make a track fro...	2026-11-1...	2026-11-14 17...	2026-11-13...	30	200.00	OPEN	true	PENDING_APPROVAL
13	4	9	3	Photography Exhibition	Still being planne...	2026-11-1...	2026-11-19 18...	2026-11-18...	60	0.00	OPEN	false	DRAFT
14	3	8	5	Late Night Concert	Evening concert.	2026-11-0...	2026-11-06 01...	2026-11-04...	300	0.00	OPEN	false	REJECTED
15	3	8	3	Rooftop Jam	Cancelled becau...	2026-10-3...	2026-10-30 20...	2026-10-29...	30	0.00	OPEN	false	CANCELLED

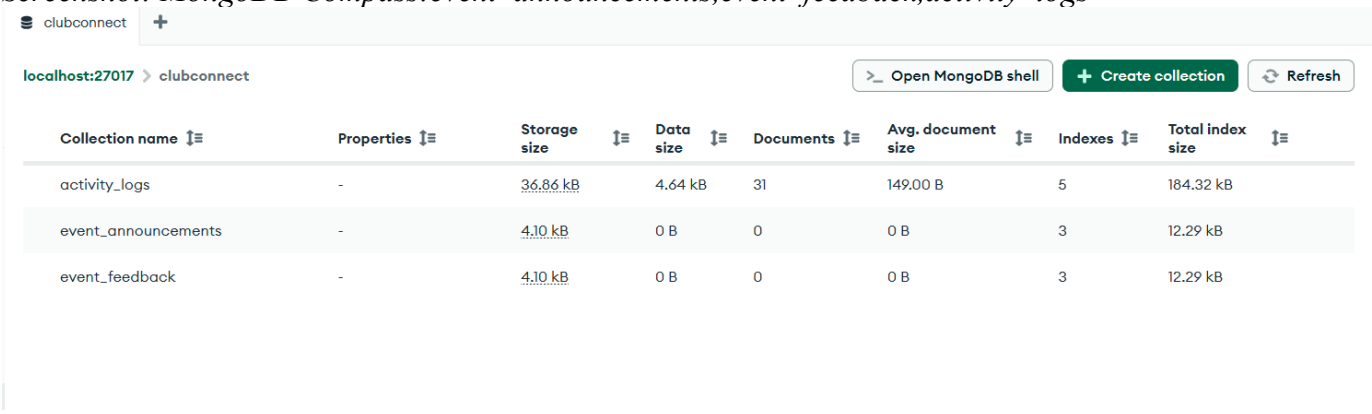
Screenshot: Exclusion-constraint error when approving overlapping events



Screenshot: Last-seat race: student B rejected (04_advanced.sql F2)



Screenshot: MongoDB Compass:event announcements,event feedback,activity logs



Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
activity_logs	-	36.86 kB	4.64 kB	31	149.00 B	5	184.32 kB
event_announcements	-	4.10 kB	0 B	0	0 B	3	12.29 kB
event_feedback	-	4.10 kB	0 B	0	0 B	3	12.29 kB

9. Next.js Project Tree Structure

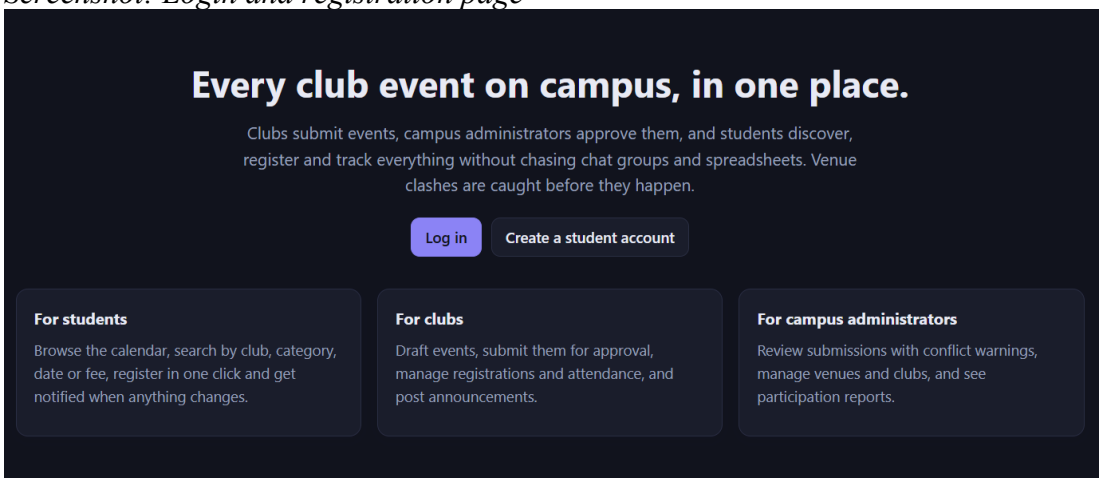
clubconnect/
- .env.example (no secrets committed)
- package.json, next.config.mjs, tsconfig.json
- db/
  - migrations/ 001_schema, 002_functions_triggers, 003_views_indexes, 004_roles_rls, 005_session_check, 006_student_cancelled_visibility
  - demo/ 01_ddl, 02_dml, 03_queries, 04_advanced, 05_rbac
- scripts/ migrate, seed, mongo-init, mongo-seed, test-db, test-api, test-mongo
- src/
  - lib/ db.ts, mongo.ts, mongo-schema.mjs, mongo-pipelines.mjs, auth.ts, session.ts, guards.ts, validation.ts, ratelimit.ts, queries.ts, reports.ts
  - components/ Shell, EventForm, CalendarView, DataTable, NotificationsView, ProfileView …
  - app/
    - login, register
    - student/ events, calendar, registrations, clubs, history, notifications, profile
    - club/ events (new, [id]), calendar, notifications, profile
    - campus/ approvals, events, conflicts, clubs, venues, categories, reports, activity
    - api/ auth, events, registrations, clubs, venues, categories, calendar, notifications, announcements, feedback, reports, activity, staff

Application architecture

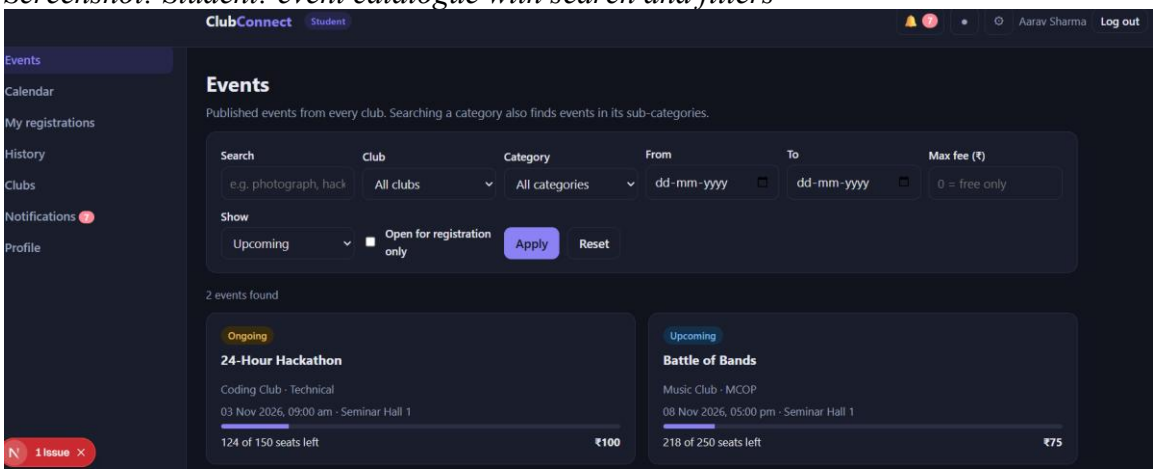
Browser (Next.js pages) → Route Handlers (/api/*, Zod validation, session check) → PostgreSQL via pg using the least-privilege login role clubconnect_app and SET LOCAL ROLE per request (student / club admin / campus admin), and MongoDB via the official driver for announcements, feedback and logs.

10. Sample Screenshots of Application Code and Demo

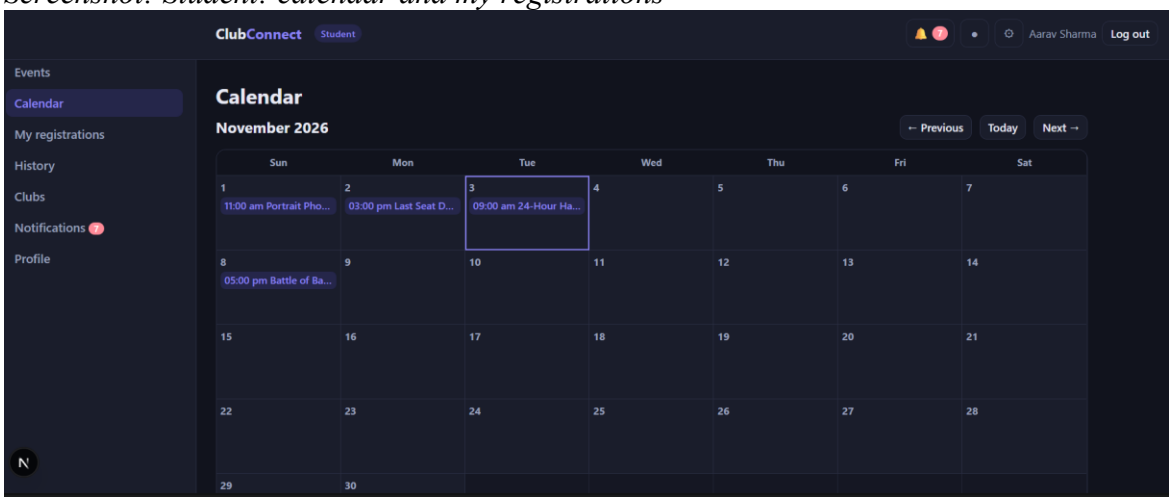
Screenshot: Login and registration page



Screenshot: Student: event catalogue with search and filters



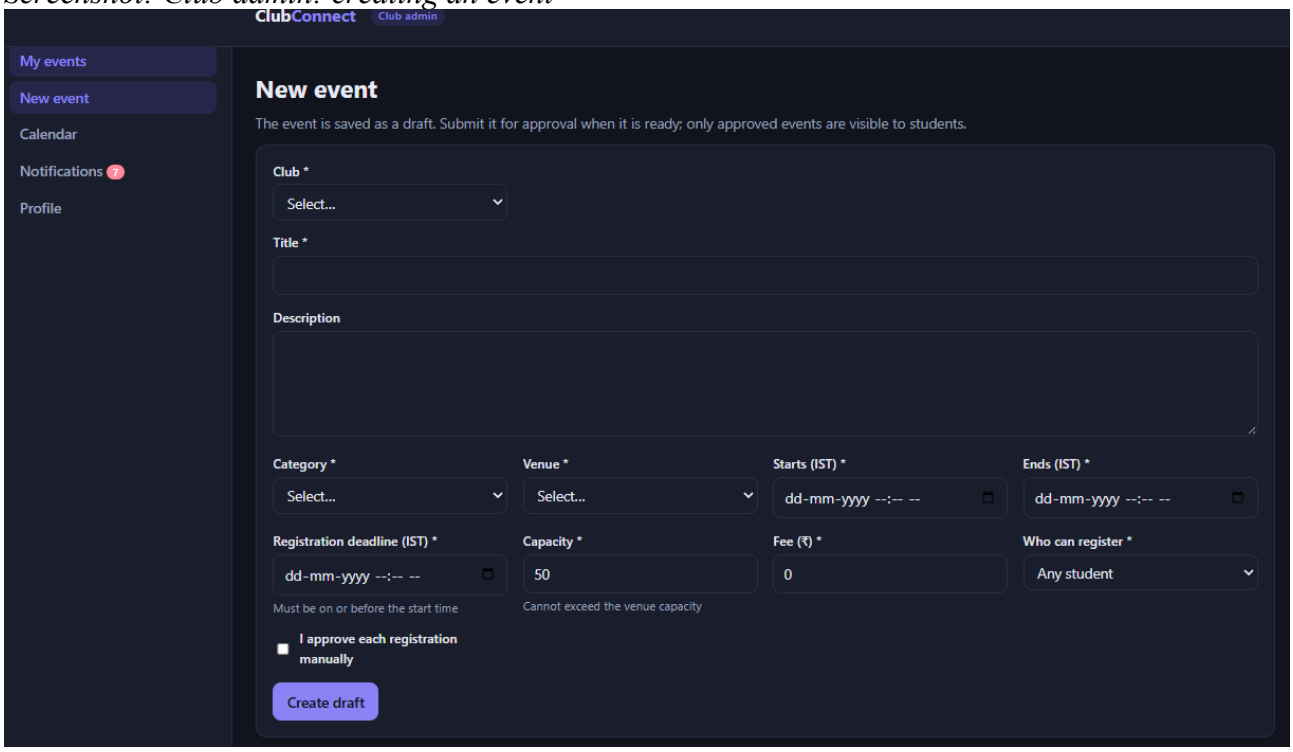
Screenshot: Student: calendar and my registrations



Screenshot: Club admin: my events page



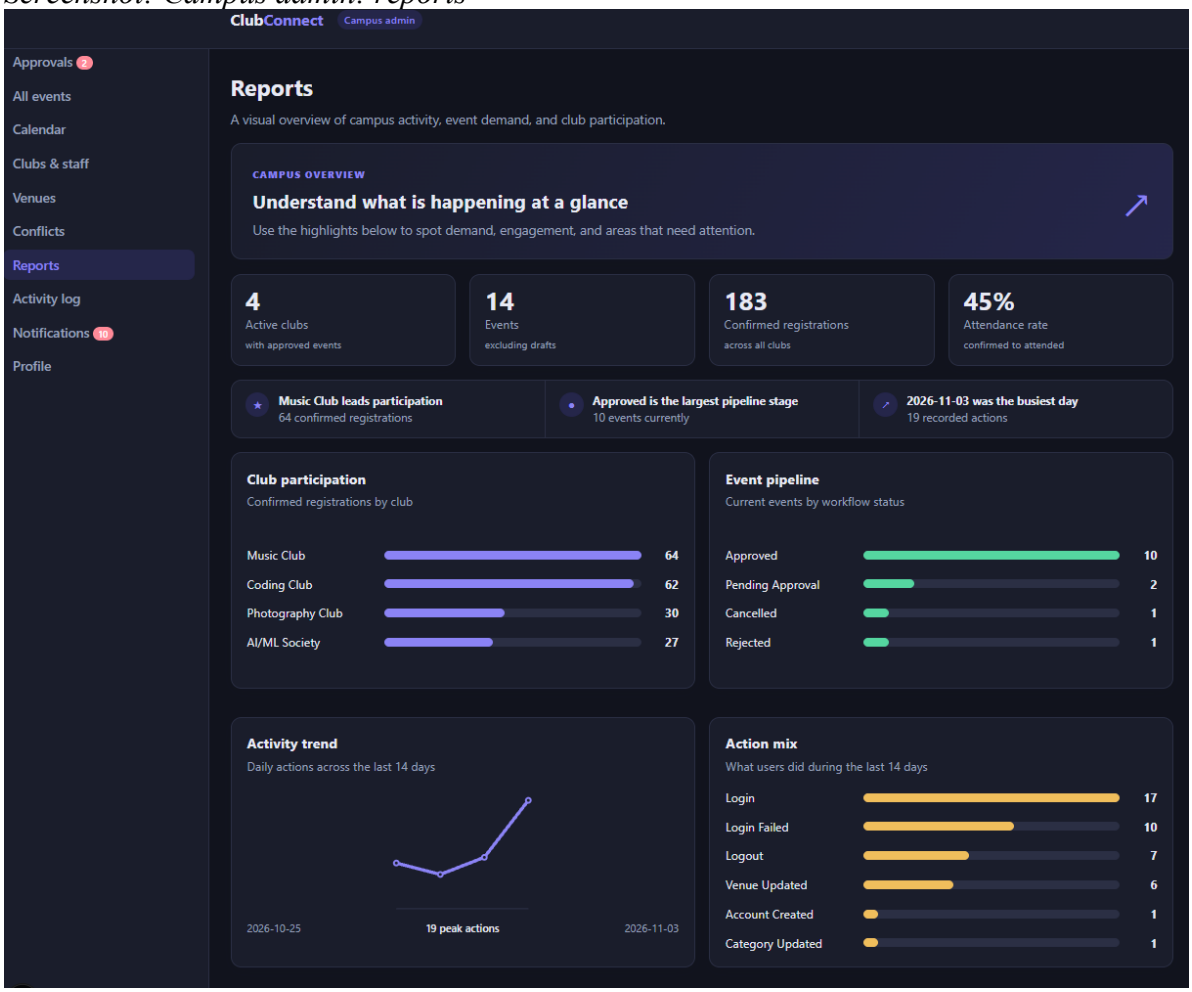
Screenshot: Club admin: creating an event



Screenshot: Campus admin: approvals and venue conflicts



Screenshot: Campus admin: reports



11. Evaluation Rubrics – Completion Status

#	Criteria	Distribution	Status
1	Database Design	ER diagram, relational schema with keys, relationships and constraints	Completed
2	Database Creation	SQL DDL with constraints, MongoDB creation, technology justification	Completed
3	SQL Operations and Queries	DML, nested/correlated/recursive queries, joins	Completed
4	Advanced Database Concepts	Functions, triggers, views, CTE, indexes, full-text search, transactions, RLS; MongoDB CRUD and aggregation	Completed
5	Web Application and Database Integration	Next.js interface; backend/API and database integration	Completed
6	Authentication and Authorization	Login (bcrypt, signed sessions); three roles with database-level RBAC	Completed

Individual contribution

Team member	Contribution
Dhriti Kamani (PES1UG24AM085)	MongoDB design,SQL schema, UI, reports, integration, testing
Disha Lokesh (PES1UG24AM093)	ER design, Next.js interface, triggers/functions,API

12. Conclusion and Future Enhancements

ClubConnect shows a clear link between database concepts, design, implementation and a working web application. Strict data (events, registrations, approvals) is kept in PostgreSQL where constraints, triggers, transactions and row-level security protect integrity, while flexible content (announcements, feedback, logs) lives in MongoDB. The core workflow – club submits an event, administrator approves, students see it on the calendar, student registers, club processes the registration and the system records the result – is fully supported.

Limitation: the MongoDB validators and TTL index should be verified on a live MongoDB server using scripts/test-mongo.mjs.

Future enhancements: online payments and refunds, QR-code attendance, e-mail/SMS reminders, semantic event recommendation using vector embeddings (pgvector), and a mobile app.